

Block 3a: Praxis I — Execution, Testing, Frontend

Workshop: Agentic Engineering
Thomas Hein — Dataciders

Execution: Den Agenten steuern

Kernprinzip: Agent schlaegt vor, Entwickler entscheidet.

Zwei Phasen — keine wird uebersprungen:

1. **Planung** — Erst Grobplanung, dann Feinplanung
2. **Execution** — Task fuer Task, mit Akzeptanzkriterien

Kein Schritt ohne explizite Bestaetigung.

Planung und Execution werden kuenftig als separate Skills bereitgestellt.

Phase 1: Planung

Grobplanung — Richtung klaeren, noch kein Code:

- Betroffene Bereiche & vorgeschlagene Reihenfolge
- Offene Designentscheidungen (mit Optionen zur Auswahl)
- Entwickler entscheidet → Akzeptanzkriterium bestaetigt

Feinplanung — auf Code-Ebene pro Task (TDD-orientiert):

- Welche Dateien werden bearbeitet? Vorher/Nachher-Vergleich
- Welche Tests zuerst? (Red → Green → Refactor)
- Akzeptanzkriterien fuer diesen Task

Feinplan bestaetigt → Execution startet

Phase 2: Execution

Immer in einem Git Worktree — parallel arbeiten auf einer Maschine.

```
git worktree add .worktrees/<feature> -b feature/<feature>
```

Pro Task:

- Agent kuendigt an → Entwickler bestaetigt
- Agent fuehrt aus nach TDD (Red → Green → Refactor)
- Entwickler prueft gegen Akzeptanzkriterien → Commit

Nach jedem 3. Task: Checkpoint — "Sind wir noch auf Kurs?"

Wo der Agent IMMER stoppen soll

- ⚠️ Neue Abhaengigkeiten einfuehren
- ⚠️ Architektur-Entscheidungen
- ⚠️ DB-Schema-Aenderungen
- ⚠️ Oeffentliche API-Aenderungen
- ⚠️ Unklare oder mehrdeutige Anforderungen

Diese Punkte gehoeren in eure AGENTS.md!

Quellen & Weiterlesen — Execution Workflow

- [Dev.to — The AI Coding Workflow That Actually Works: Separate Planning from Execution](#) — Warum Planning und Execution getrennt werden sollten
- [InfoQ — From Prompts to Production: a Playbook for Agentic Development](#) — Praxisleitfaden fuer agentic Development-Workflows
- [Thoughtworks — Preparing your team for the agentic SDLC](#) — Team-Vorbereitung fuer agentic Software Development
- [GitHub Blog — How to build reliable AI workflows with agentic primitives](#) — Zuverlässige AI-Workflows mit Context Engineering
- [Knostic — AI Coding Agent Governance Policies That Work](#) — Governance-Regeln fuer AI-Coding-Agents

Testing: Die Kontrolle behalten

Das Kernproblem

*Der Agent passt den **Test** an, statt den **Code** zu fixen.
→ Regression wird unsichtbar.*

Loesung: Tests zuerst, dann Implementierung (TDD)

Harte Regeln fuer Tests

1. **Tests zuerst** — erst der Test, dann der Code
2. **Tests sind heilig** — nicht aendern, um gruen zu werden
3. **Einzelnen durchgehen** — bei Fehlschlaegen: Regression oder erwartete Aenderung?
4. **Ohne Tests kein Merge**

Diese Regeln gehoeren als "Hard Facts" in eure AGENTS.md!

Feedbackschleifen

Ebene	Was	Wann
Unit Tests	Einzelne Funktionen	Bei jedem Task
Integration	Modul-Zusammenspiel	Nach Feature
Akzeptanz	Fachliche Korrektheit	Vor Merge
E2E (Playwright)	UI-Verhalten automatisiert	Optional, bei UI

Je frueher der Fehler gefunden wird, desto guenstiger die Korrektur.

Quellen & Weiterlesen — Testing und TDD

- [GitHub Blog — Test-driven development \(TDD\) with GitHub Copilot](#) — TDD-Grundlagen und Copilot-Workflow
- [Nimble Approach — How to Use TDD for better AI coding outputs](#) — TDD als Qualitätsversicherung bei AI-generiertem Code
- [Dev.to — How I Validate Quality When AI Agents Write My Code](#) — Praxisbericht zur Qualitätskontrolle bei Agent-generiertem Code

Frontend-Entwicklung mit Agenten

Wie beschreibe ich eine UI passgenau?

Methode	Aufwand	Präzision	Empfehlung
Screenshots + Paint	Niedrig	Mittel	Alltag
Figma / Penpot	Hoch	Hoch	Neue Komponenten
Design Tokens (JSON)	Mittel	Hoch	Ab mittlerer Projektgröße
Storybook (React / WC)	Mittel	Sehr hoch	Komponenten-Bibliotheken

Pragmatisch: Screenshots + Annotationen reichen meistens!

Storybook: Stories als lebende Spezifikation für den Agenten nutzen.

Quellen & Weiterlesen — Frontend mit Agenten

- [Style Dictionary \(Amazon\)](#) — Build-Tool fuer Design Tokens: JSON → CSS/JS/native, framework-agnostisch
- [Tokens Studio fuer Figma](#) — Design Tokens direkt in Figma erstellen und als JSON exportieren
- [Penpot Docs — Design Tokens](#) — Design Tokens nativ in Penpot verwalten und exportieren
- [W3C Design Tokens Format](#) — Offener Standard fuer Interoperabilitaet zwischen Design-Tools
- [Figma Dev Mode](#) — Figma-Export fuer Entwickler, Handoff und Inspektion
- [Penpot — Open Source Design Tool](#) — Kostenlose Open-Source-Alternative zu Figma, self-hostable
- [Storybook — UI Component Explorer](#) — Komponenten isoliert entwickeln, dokumentieren und als Agent Spezifikation nutzen